

SSD_AUDIO usage reference

1. AI (audio input) use reference

1.1. Input interface types supported by AI (using SSD20X series)

AI currently supports amic, dmic, i2s, linein, and the corresponding specifications are as follows:

Interface Type	maximum channel	Sampling Rate	Sampling bit depth	channel
amic	1	8/16/32/48KHz	16bit	STEREO/MONO
dmic	4	8/16/32/48KHz	16bit	STEREO/MONO
I2s	Standard 2-way (does not support TDM)	8/16/32/48 KHz (mclk not supported)	16bit	STEREO/MONO
linein	3	8/16/32/48KHz	16bit	STEREO/MONO

1.2. AI-supported device ID

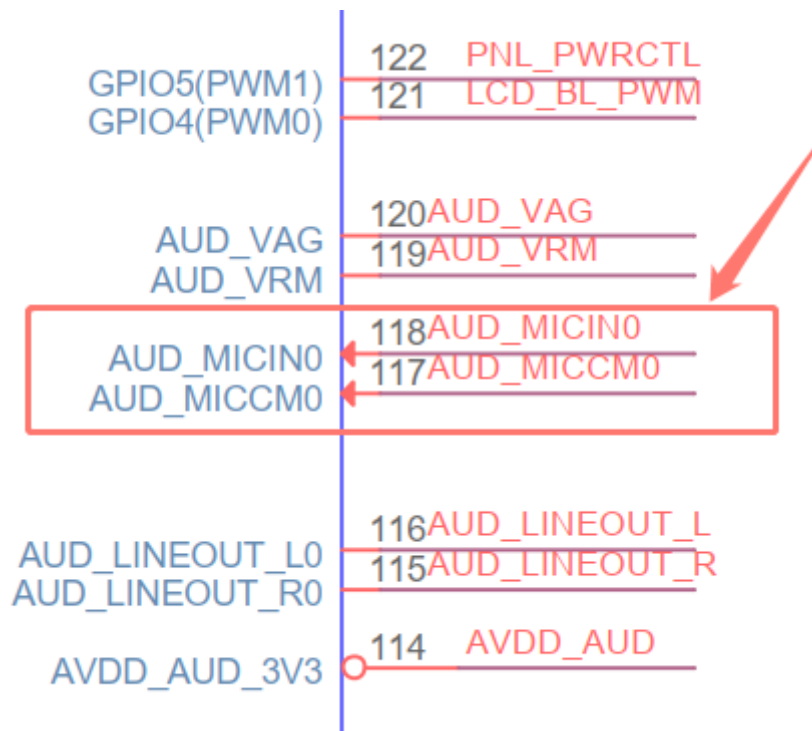
When AI is initialized, different device IDs are passed in to represent the use of different interface types, as follows:

Device ID	Interface Type
AI_DEV_ID_AMIC (0)	Amic
AI_DEV_ID_DMIC (1)	Dmic
AI_DEV_ID_I2S_RX (2)	I2S RX
AI_DEV_ID_LINE_IN (3)	Line in
AI_DEV_ID_AMIC_AND_I2S_RX (4)	Amic + I2S RX
AI_DEV_ID_DMIC_AND_I2S_RX (5)	Dmic + I2S RX
AI_DEV_ID_AMIC_AND_DMIC (6)	2Amic + 4Dmic

1.3. Amic usage scenarios

One channel amic is mainly used in the usage scenario of building intercom, which is used to collect one channel sound to realize the function of indoor and outdoor intercom.

The following is an example of the pin of the public version of Amic:



The corresponding device ID and channel index are:

Pin	Device ID
AI_DEV_ID_AMIC	0

The reference to use Demo is as follows:

```

...MI_AUDIO_Attr_t stAiSetAttr;
...
...memset(&stAiSetAttr, 0x0, sizeof(MI_AUDIO_Attr_t));
...
...//int AI
...stAiSetAttr.eBitwidth = E_MI_AUDIO_BIT_WIDTH_16;
...stAiSetAttr.eSamplerate = eAiSampleRate;
...stAiSetAttr.eSoundmode = E_MI_AUDIO_SOUND_MODE_MONO;
...stAiSetAttr.u32ChnCnt = 1;
...stAiSetAttr.u32PtNumPerFrm = stAiSetAttr.eSamplerate / 16;

...ExecFunc(MI_AI_SetPubAttr(AiDevId, &stAiSetAttr), MI_SUCCESS);
...ExecFunc(MI_AI_GetPubAttr(AiDevId, &stAiGetAttr), MI_SUCCESS);
...ExecFunc(MI_AI_Enable(AiDevId), MI_SUCCESS);

...ExecFunc(MI_AI_SetVqeVolume(AiDevId, u32ChnIdx, s32AiVolume), MI_SUCCESS);
...ExecFunc(MI_SYS_SetChnOutputPortDepth(&stAiChnOutputPort0[u32ChnIdx], USER_BUF_DEPTH, TOTAL_BUF_DEPTH), MI_SUCCESS);
...ExecFunc(MI_AI_EnableChn(AiDevId, u32ChnIdx), MI_SUCCESS);

...//Get Ai Frame
...MI_AUDIO_Frame_t stAiChFrame;
...MI_AUDIO_AecFrame_t stAecFrame;

...memset(&stAiChFrame, 0, sizeof(MI_AUDIO_Frame_t));
...memset(&stAecFrame, 0, sizeof(MI_AUDIO_AecFrame_t));

...while(FALSE == bAiExit)
...{
...    s32Ret = MI_AI_GetFrame(AiDevId, u32ChnIdx, &stAiChFrame, &stAecFrame, 1);
...    if (MI_SUCCESS == s32Ret)
...    {
...        write(priv->s32Fd, stAiChFrame.apVirAddr[0], stAiChFrame.u32Len[0]);
...        MI_AI_ReleaseFrame(AiDevId, u32ChnIdx, &stAiChFrame, NULL);
...    }
...    else
...    {
...        printf("Dev%dChn%d get frame failed!!error:0x%x\n", AiDevId, u32ChnIdx, s32Ret);
...    }
...}
...
...

```

1.4. Dmic usage scenarios

1.4.1 Configure pin to dmic mode

Before using dmic, you need to confirm with the hardware which set of pins to use, and then configure digmic-padmux to the corresponding Mode in the infinity2m.dtsi and padmux files. For example, the public version uses PAD_HDMITX_SCL (D1) & PAD_HDMITX_SDA (D0) & PAD_HDMITX_HPDP (CLK)

As follows, find the corresponding *.dtsi file and find the sound node configuration as follows:

```

sound {
    compatible = "sstar, audio";
    reg = <0x1F000000 0x1000000>;
    interrupts=<GIC_SPI INT_IRQ_BACH IRQ_TYPE_LEVEL_HIGH>;
    playback-volume-level=<64>;    //0~94
    capture-volume-level=<64>;
    // micin-pregain-level=<1>;    //0~3
    micin-pregain-level=<0>;    //0~3
    micin-gain-level=<3>;    //0~7
    linein-gain-level=<2>;    //0~7
    amp-gpio = <PAD_FUART_RX 1>;
    clocks = <&CLK_upll_384m>;
    // playback-dma-buffer=<98304>; //512(ms)*48(kHz)*2(ch)*2(16bits)
    // capture-dma-buffer=<122880>; //640(ms)*48(kHz)*2(ch)*2(16bits)
    digmic-padmux = <2>;
    i2s-padmux = <2>;
    keep-i2s-clk = <0>;
    status = "ok";
};

```

At the same time, find the corresponding *padmux.dtsi file and configure the corresponding padmux mode

```

//<PAD_HSYNC_OUT >,
//<PAD_VSYNC_OUT >,
<PAD_HDMITX_SCL      PINMUX_FOR_DMIC_MODE_2      MDRV_PUSE_DMIC_D1 >,
<PAD_HDMITX_SDA      PINMUX_FOR_DMIC_MODE_2      MDRV_PUSE_DMIC_D0 >,
<PAD_HDMITX_HPD      PINMUX_FOR_DMIC_MODE_2      MDRV_PUSE_DMIC_CLK >,
<PAD_HSYNC_OUT      PINMUX_FOR_IDAC_MODE          MDRV_PUSE_IDAC_HSYNC>,
<PAD_VSYNC_OUT      PINMUX_FOR_IDAC_MODE          MDRV_PUSE_IDAC_VSYNC>;
//<PAD_SATA_GPIO >;

```

1.4.2 4 dmic usage scenarios

When Dmic is in use, regardless of whether it is 2dmic or 4dmic, the device ID used is AI_DEV_ID_DMIC (1), and ChannelCount is used to determine how many channels of audio need to be collected.

The reference demo is used as follows:

```

...MI_AUDIO_Attr_t.stAiSetAttr;
...
...memset(&stAiSetAttr, 0x0, sizeof(MI_AUDIO_Attr_t));
...
...//int-AI
...stAiSetAttr.eBitwidth = E_MI_AUDIO_BIT_WIDTH_16;
...stAiSetAttr.eSamplerate = eAiSampleRate;
...stAiSetAttr.eSoundmode = E_MI_AUDIO_SOUND_MODE_MONO;
...stAiSetAttr.u32ChnCnt = 4;
...stAiSetAttr.u32PtNumPerFrm = stAiSetAttr.eSamplerate / 16;

...ExecFunc(MI_AI_SetPubAttr(AiDevId, &stAiSetAttr), MI_SUCCESS);
...ExecFunc(MI_AI_GetPubAttr(AiDevId, &stAiGetAttr), MI_SUCCESS);
...ExecFunc(MI_AI_Enable(AiDevId), MI_SUCCESS);

...for(u32ChnIdx = 0; u32ChnIdx < u32ChnCnt; u32ChnIdx++)
...{
...    ExecFunc(MI_AI_SetVqeVolume(AiDevId, u32ChnIdx, s32AiVolume), MI_SUCCESS);
...    ExecFunc(MI_SYS_SetChnOutputPortDepth(&stAiChnOutputPort0[u32ChnIdx], USER_BUF_DEPTH, TOTAL_BUF_DEPTH), MI_SUCCESS);
...    ExecFunc(MI_AI_EnableChn(AiDevId, u32ChnIdx), MI_SUCCESS);
...}

...//Open 4 thread to get AI frame
...//Thread #1 & #2 & #3 & #4 use channel index 0 & 1 & 2 & 3
...MI_AUDIO_Frame_t stAiChFrame;
...MI_AUDIO_AecFrame_t stAecFrame;

...memset(&stAiChFrame, 0, sizeof(MI_AUDIO_Frame_t));
...memset(&stAecFrame, 0, sizeof(MI_AUDIO_AecFrame_t));

...while(FALSE == bAiExit)
...{
...    s32Ret = MI_AI_GetFrame(AiDevId, u32ChnIdx, &stAiChFrame, &stAecFrame, -1);
...    if(MI_SUCCESS == s32Ret)
...    {
...        write(priv->s32Fd, stAiChFrame.apVirAddr[0], stAiChFrame.u32Len[0]);
...        MI_AI_ReleaseFrame(AiDevId, u32ChnIdx, &stAiChFrame, NULL);
...    }
...    else
...    {
...        printf("Dev%dChn%d get frame failed!!!error:0x%x\n", AiDevId, u32ChnIdx, s32Ret);
...    }
...}
} //end if dmic

```

1.5. I2s RX usage scenarios

1.5.1 I2s rx pin configuration

使用i2s rx前，需要先跟硬件确认使用的是哪组pin，然后在对应的padmux里面配置成对应的Mode，例如公版如果使用的是PAD_SD_D0 & PAD_SD_CLK & PAD_SD_CMD & PAD_SD_D3 这组mode，对应的配置如下：

找到对应的dtsi文件中的sound节点：

```

sound {
    compatible = "sstar, audio";
    reg = <0x1F000000 0x1000000>;
    interrupts = <GIC_SPI INT_IRQ_BACH IRQ_TYPE_LEVEL_HIGH>;
    playback-volume-level = <64>; //0~94
    capture-volume-level = <64>;
    // micin-pregain-level = <1>; //0~3
    micin-pregain-level = <0>; //0~3
    micin-gain-level = <3>; //0~7
    linein-gain-level = <2>; //0~7
    amp-gpio = <PAD_FUART_RX 1>;
    clocks = <&CLK_upll_384m>;
    // playback-dma-buffer = <98304>; //512(ms)*48(kHz)*2(ch)*2(16bits)
    // capture-dma-buffer = <122880>; //640(ms)*48(kHz)*2(ch)*2(16bits)
    digmic-padmux = <2>;
    i2s-padmux = <3>;
    keep-i2s-clk = <0>;
    status = "ok";
};

```

找到对应的*padmux.dtsi文件，配置以下padmux：

<PAD_SD_D0	PINMUX_FOR_I2S_MODE_3	MDRV_PUSE_I2S_WCK> ,
<PAD_SD_CLK	PINMUX_FOR_I2S_MODE_3	MDRV_PUSE_I2S_BCK> ,
<PAD_SD_CMD	PINMUX_FOR_I2S_MODE_3	MDRV_PUSE_I2S_SDI> ,
<PAD_SD_D3	PINMUX_FOR_I2S_MODE_3	MDRV_PUSE_I2S_SD0> ,

使用的pin和mode的对应关系参考硬件资料的HW CheckList。

1.5.2 普通两路i2s使用场景

普通两路i2s rx的使用Demo参考如下：

```

..MI_AUDIO_Attr_t·stAiSetAttr;
..memset(&stAiSetAttr, 0x0, sizeof(MI_AUDIO_Attr_t));
..
..//int·AI
..stAiSetAttr.eBitwidth.=·E_MI_AUDIO_BIT_WIDTH_16;
..stAiSetAttr.eSamplerate.=·eAiSampleRate;
..stAiSetAttr.eSoundmode.=·E_MI_AUDIO_SOUND_MODE_MONO;
..stAiSetAttr.u32ChnCnt = 2;
..stAiSetAttr.u32PtNumPerFrm.=·stAiSetAttr.eSamplerate./·16;

..stAiSetAttr.eWorkmode.=·E_MI_AUDIO_MODE_I2S_MASTER;
..stAiSetAttr.WorkModeSetting.stI2sConfig.bSyncClock.=·FALSE;·//FALSE:6·wire·mode·,·TRUE:·4·wire·mode
..stAiSetAttr.WorkModeSetting.stI2sConfig.eFmt.=·E_MI_AUDIO_I2S_FMT_I2S_MSB;
..stAiSetAttr.WorkModeSetting.stI2sConfig.eMclk.=·E_MI_AUDIO_I2S_MCLK_0;·//接受/发送的mclk
..stAiSetAttr.WorkModeSetting.stI2sConfig.u32TdmSlots.=·2;·//codec通过i2s实际送过来的数据channel·num
..stAiSetAttr.WorkModeSetting.stI2sConfig.eI2sBitWidth.=·E_MI_AUDIO_BIT_WIDTH_16;

..ExecFunc(MI_AI_SetPubAttr(AiDevId, &stAiSetAttr), MI_SUCCESS);
..ExecFunc(MI_AI_GetPubAttr(AiDevId, &stAiGetAttr), MI_SUCCESS);
..ExecFunc(MI_AI_Enable(AiDevId), MI_SUCCESS);
..for·(u32ChnIdx.=·0;·u32ChnIdx.<·u32ChnCnt;·u32ChnIdx++)
..{
..    ExecFunc(MI_AI_SetVqeVolume(AiDevId, u32ChnIdx, s32AiVolume), MI_SUCCESS);
..    ExecFunc(MI_SYS_SetChnOutputPortDepth(&stAiChnOutputPort0[u32ChnIdx], USER_BUF_DEPTH, TOTAL_BUF_DEPTH), MI_SUCCESS);
..    ExecFunc(MI_AI_EnableChn(AiDevId, u32ChnIdx), MI_SUCCESS);
..}
..//Open·2·thread·To·Get·Ai·frame
..//Thread·#1·&·#2·use·channel·index·0·&·1·
..MI_AUDIO_Frame_t·stAiChFrame;
..MI_AUDIO_AecFrame_t·stAecFrame;|
..memset(&stAiChFrame, 0, sizeof(MI_AUDIO_Frame_t));
..memset(&stAecFrame, 0, sizeof(MI_AUDIO_AecFrame_t));

..while(FALSE==·bAiExit)
..{
..    s32Ret.=·MI_AI_GetFrame(AiDevId, u32ChnIdx, &stAiChFrame, &stAecFrame, -1);
..    if·(MI_SUCCESS==·s32Ret)
..    {
..        write(priv->s32Fd, stAiChFrame.apVirAddr[0], stAiChFrame.u32Len[0]);
..        MI_AI_ReleaseFrame(AiDevId, u32ChnIdx, &stAiChFrame, NULL);
..    }
..    else
..    {
..        printf("Dev%dChn%d·get·frame·failed!!!error:0x%x\n", AiDevId, u32ChnIdx, s32Ret);
..    }
..}

```

1.6. Line In的使用场景

Line in的使用方式跟amic一样，只是设定Gain的方式不一样,具体参考AI [./miapi/mi_ai.html] API文档说明。

2. AO(audio output)使用场景参考

2.1. AO支持的输出接口类型

接口类型	最大 channel	采样率	采样深度	声道
Line out	2	8/11.025/12/16/22.05/32/44.1/48KHz	16bit	STEREO
I2s	2	8/16/32/48 KHz(不支持mclk)	16bit	STEREO

2.2. AO支持的Device ID

Device ID	接口类型
AO_DEV_ID_LINE_OUT (0)	两路Lineout
AO_DEV_ID_I2S_TX (1)	I2s
AO_DEV_ID_HDMI (2)	HDMI(仅SSD203支持)
AO_DEV_ID_HDMI_AND_LINEOUT (3)	HDMI + LINE OUT (仅SSD203支持)

2.3. Lineout使用场景

2.3.1 Lineout Amp的配置说明

SSD20X持两路Line out左右声道输出，但不支持单路属性控制操作（如不支持单独音量调节）

根据硬件layout可以按照以下方式配置amp_gpio

- 1. 打开kernel\arch\arm\boot\dts\infinity2m.dtsi，修改sound设备树节点amp-gpio

如下图，设置PAD_GPIO12/PAD_FUART_RX为amp_gpio的左右声道控制pin

```

sound {
    compatible = "sstar,audio";
    reg = <0x1F000000 0x1000000>;
    interrupts=<GIC_SPI INT_IRQ_BACH IRQ_TYPE_LEVEL_HIGH>;
    playback-volume-level=<64>; //0~94
    capture-volume-level=<64>;
    // micin-pregain-level=<1>; //0~3
    micin-pregain-level=<0>; //0~3
    micin-gain-level=<3>; //0~7
    linein-gain-level=<2>; //0~7
    amp-gpio = <PAD_GPIO12 1 PAD_FUART_RX 1>;
    clocks = <&CLK_upll_384m>;
    // playback-dma-buffer=<98304>; //512 (ms) *48 (kHz) *2 (ch) *2 (16bits)
    // capture-dma-buffer=<122880>; //640 (ms) *48 (kHz) *2 (ch) *2 (16bits)
    digmic-padmux = <2>;
    i2s-padmux = <3>;
    keep-i2s-clk = <0>;
    status = "ok";
};

```

2. 打开kernel\arch\arm\boot\dts\pioneer3-ssc020a-s01a-demo-camera-padmux.dtsi (根据实际情况打开config使用的*_padmux.dtsi)

如下图，分别设置PAD_SR_IO15/PAD_SR_IO16为

MDRV_PUSE_AIO_AMP_PWL/MDRV_PUSE_AIO_AMP_PWR

<PAD_GPIO12	PINMUX_FOR_GPIO_MODE	MDRV_PUSE_AIO_AMP_PWL> ,
<PAD_FUART_RX	PINMUX_FOR_GPIO_MODE	MDRV_PUSE_AIO_AMP_PWR> ,
<PAD_GPIO13	PINMUX_FOR_GPIO_MODE	MDRV_PUSE_I2C1_DEV_IRQ > ,

2.3.2 一路lineout使用场景

使用Lineout支持左右声道，但是只有一路dev，如果需要播放立体左右声道，需要音频文件本身就左右声道组合好通过STEREO MODE播放(不支持单独喂数据)

参考使用Demo如下：

```

..//init.ao
..MI_AUDIO_Attr_t stAoSetAttr, stAoGetAttr;

..memset(&stAoSetAttr, 0x0, sizeof(MI_AUDIO_Attr_t));
..
..stAoSetAttr.eBitwidth = E_MI_AUDIO_BIT_WIDTH_16;
..stAoSetAttr.u32PtNumPerFrm = MI_AUDIO_SAMPLE_PER_FRAME;
..stAoSetAttr.eSoundmode = E_MI_AUDIO_SOUND_MODE_MONO;
..stAoSetAttr.u32ChnCnt = 1;

..stAoSetAttr.eSamplerate = eAoInSampleRate;

..ExecFunc(MI_AO_SetPubAttr(AoDevId, &stAoSetAttr), MI_SUCCESS);
..ExecFunc(MI_AO_GetPubAttr(AoDevId, &stAoGetAttr), MI_SUCCESS);
..ExecFunc(MI_AO_Enable(AoDevId), MI_SUCCESS);
..ExecFunc(MI_AO_EnableChn(AoDevId, AoChn), MI_SUCCESS);

..ExecFunc(MI_AO_SetVolume(AoDevId, 0, s32AoVolume, E_MI_AO_GAIN_FADING_OFF), MI_SUCCESS);
..ExecFunc(MI_AO_GetVolume(AoDevId, 0, &s32AoGetVolume), MI_SUCCESS);
..//send frame
..MI_AUDIO_Frame_t stAoSendFrame;
..while (FALSE == bAoExit)
..{
..    memset(&stAoSendFrame, 0x0, sizeof(MI_AUDIO_Frame_t));
..    stAoSendFrame.u32Len[0] = BufLen; //buflen must >= bitwidth!
..    stAoSendFrame.apVirAddr[0] = u8TempBuf;
..    stAoSendFrame.apVirAddr[1] = NULL;

..    do{
..        s32Ret = MI_AO_SendFrame(AoDevId, AoChn, &stAoSendFrame, -1);
..    }while(s32Ret == MI_AO_ERR_NOBUF);

..    if (s32Ret != MI_SUCCESS)
..    {
..        printf("[Warning]: MI_AO_SendFrame fail, error is 0x%x: \n", s32Ret);
..    }
..}

```

2.4. I2s Tx使用场景

2.4.1 I2s Tx pin脚配置

使用i2s rx前，需要先跟硬件确认使用的是哪组pin，然后在对应的padmux里面配置成对应的Mode，例如公版如果使用的是PAD_SD_D0 & PAD_SD_CLK & PAD_SD_CMD && PAD_SD_D3 这组mode，对应的配置如下：

```

sound {
    compatible = "sstar, audio";
    reg = <0x1F000000 0x1000000>;
    interrupts = <GIC_SPI INT_IRQ_BACH IRQ_TYPE_LEVEL_HIGH>;
    playback-volume-level = <64>; //0~94
    capture-volume-level = <64>;
    // micin-pregain-level = <1>; //0~3
    micin-pregain-level = <0>; //0~3
    micin-gain-level = <3>; //0~7
    linein-gain-level = <2>; //0~7
    amp-gpio = <PAD_FUART_RX 1>;
    clocks = <&CLK_upll_384m>;
    // playback-dma-buffer = <98304>; //512(ms)*48(kHz)*2(ch)*2(16bits)
    // capture-dma-buffer = <122880>; //640(ms)*48(kHz)*2(ch)*2(16bits)
    digmic-padmux = <2>;
    i2s-padmux = <3>;
    keep-i2s-clk = <0>;
    status = "ok";
};

```

找到对应的*padmux.dtsi文件，配置以下padmux：

<PAD_SD_D0	PINMUX_FOR_I2S_MODE_3	MDRV_PUSE_I2S_WCK> ,
<PAD_SD_CLK	PINMUX_FOR_I2S_MODE_3	MDRV_PUSE_I2S_BCK> ,
<PAD_SD_CMD	PINMUX_FOR_I2S_MODE_3	MDRV_PUSE_I2S_SDI> ,
<PAD_SD_D3	PINMUX_FOR_I2S_MODE_3	MDRV_PUSE_I2S_SD0> ,

使用的pin和mode的对应关系参考硬件资料的HW CheckList。

2.4.2 普通两路i2s tx使用场景

参考使用Demo如下：

```

...memset(&stAoSetAttr, 0x0, sizeof(MI_AUDIO_Attr_t));

...if (stWavHeaderInput.wave.wChannels == 2)
...{
...    stAoSetAttr.eSoundmode = E_MI_AUDIO_SOUND_MODE_STEREO;
...    u32PhyChnNum = 2;
...}
...else if (stWavHeaderInput.wave.wChannels == 1)
...{
...    stAoSetAttr.eSoundmode = E_MI_AUDIO_SOUND_MODE_MONO;
...    u32PhyChnNum = 1;
...}

...stAoSetAttr.eSamplerate = (MI_AUDIO_SampleRate_e)stWavHeaderInput.wave.dwSamplesPerSec;
...stAoSetAttr.eBitwidth = E_MI_AUDIO_BIT_WIDTH_16;
...stAoSetAttr.u32PtNumPerFrm = MI_AUDIO_SAMPLE_PER_FRAME;
...stAoSetAttr.u32ChnCnt = 1;
...
...if (AoDevId != AO_DEV_I2S_TX)
...{
...    stAoSetAttr.eWorkmode = E_MI_AUDIO_MODE_I2S_MASTER;
...    stAoSetAttr.WorkModeSetting.stI2sConfig.bSyncClock = FALSE; // 6.wire: FALSE, 4.wire: TRUE
...    stAoSetAttr.WorkModeSetting.stI2sConfig.eFmt = E_MI_AUDIO_I2S_FMT_I2S_MSB;
...    stAoSetAttr.WorkModeSetting.stI2sConfig.eMclk = E_MI_AUDIO_I2S_MCLK_0;
...    stAoSetAttr.WorkModeSetting.stI2sConfig.u32TdmSlots = 2; // i2s.tx.channel.num
...    stAoSetAttr.WorkModeSetting.stI2sConfig.eI2sBitWidth = E_MI_AUDIO_BIT_WIDTH_16;
...}

...ExecFunc(MI_AO_SetPubAttr(AoDevId, &stAoSetAttr), MI_SUCCESS);
...ExecFunc(MI_AO_GetPubAttr(AoDevId, &stAoGetAttr), MI_SUCCESS);
...ExecFunc(MI_AO_Enable(AoDevId), MI_SUCCESS);
...ExecFunc(MI_AO_EnableChn(AoDevId, AoChn), MI_SUCCESS);

...ExecFunc(MI_AO_SetVolume(AoDevId, 0, s32AoVolume, E_MI_AO_GAIN_FADING_OFF), MI_SUCCESS);

...s32NeedSize = MI_AUDIO_SAMPLE_PER_FRAME * 2 * u32PhyChnNum * stAoSetAttr.u32ChnCnt;
...s32NeedSize = s32NeedSize / ((stAoSetAttr.u32ChnCnt * 2 * u32PhyChnNum) * (stAoSetAttr.u32ChnCnt * 2 * u32PhyChnNum));

...pthread_create(&AoTid, NULL, aoSendFrame, NULL);

```

```

void* aoSendFrame(void* data)
{
...MI_S32 s32Ret = MI_SUCCESS;
...MI_AUDIO_Frame_t stAoSendFrame;

...while (FALSE == bAoExit)
...{
...    s32Ret = read(AoReadFd, &u8TempBuf, s32NeedSize);
...    if (s32Ret != s32NeedSize)
...    {
...        lseek(AoReadFd, sizeof(WaveFileHeader_t), SEEK_SET);
...        s32Ret = read(AoReadFd, &u8TempBuf, s32NeedSize);
...        if (s32Ret < 0)
...        {
...            printf("Input file does not has enough data!!!\n");
...            break;
...        }
...    }

...    memset(&stAoSendFrame, 0x0, sizeof(MI_AUDIO_Frame_t));
...    stAoSendFrame.u32Len[0] = s32Ret;
...    stAoSendFrame.apVirAddr[0] = u8TempBuf;
...    stAoSendFrame.apVirAddr[1] = NULL;
...    //printf("send data: %x, len: %d\n", stAoSendFrame.apVirAddr[0], stAoSendFrame.u32Len[0]);
...    do{
...        s32Ret = MI_AO_SendFrame(AoDevId, AoChn, &stAoSendFrame, -1);
...    }while(s32Ret == MI_AO_ERR_NOBUF);

...    if (s32Ret != MI_SUCCESS)
...    {
...        printf("[Warning]: MI_AO_SendFrame fail, error is 0x%x:\n", s32Ret);
...    }
...} // end while FALSE == bAoExit
...close(AoReadFd);
...return NULL;
} // end aoSendFrame

```